

3

HỌC MÁY  
TĂNG CƯỜNG CHO  
CÁC HỆ THỐNG MẠNG

# Giới thiệu về Deep Q-Network

Biên soạn: ThS. Phan Trung Phát  
Liên hệ: [phatpt@uit.edu.vn](mailto:phatpt@uit.edu.vn)

Tái bản lần 1 -- Tháng 03/2026

Lưu hành nội bộ

## A. MỤC TIÊU

- Nắm rõ và sử dụng được một số thành phần cơ bản của PyTorch.
- Tích hợp toàn bộ thuật toán DQN với các môi trường có sẵn trên Gymnasium và môi trường tùy chỉnh.
- Tìm hiểu về quá trình huấn luyện và đào tạo Agent, hiểu được các hyperparameters và cách tùy chỉnh để đạt được kết quả hiệu trả trên giải thuật DRL.
- Đánh giá và so sánh được chính sách được xây dựng dựa trên DQN và các policy khác.
- Tìm hiểu Stable-Baselines3 và tích hợp DQN một cách đơn giản.

## B. GIỚI THIỆU

### 1. Động lực phát triển

#### a. Sơ lược lại về Q-learning

Trong học máy tăng cường, mục tiêu của Agent là tìm ra một chính sách tối ưu. **Q-learning** thực hiện việc này bằng cách học giá trị  $q(s, a)$  — đại diện cho "chất lượng" của một hành động  $a$  tại trạng thái  $s$ . Trong phương pháp tiếp cận truyền thống, kiến thức của Agent được cụ thể hóa bằng một Ma trận (Matrix) hoặc một Bảng tra cứu (Lookup Table) và thông thường sẽ được gọi là **Q-Table**. Hãy tưởng tượng đây là một "cuốn sổ tay" ghi chép lại mọi kinh nghiệm mà Robot đã trải qua chẳng hạn.

Khi Agent tương tác với Environment: Nó sẽ quan sát trạng thái hiện tại → Thực hiện một hành động → Nhận phần thưởng và trạng thái mới → Sau đó cập nhật lại đúng ô trong Q-Table. Cấu trúc của Q-Table thông thường sẽ lưu trữ dưới cặp giá trị  $(S, A)$ , cụ thể được mô tả:

- **Hàng:** Đại diện cho mọi trạng thái có thể có ( $s \in S$ ).
- **Cột:** Đại diện cho mọi hành động có thể thực hiện ( $a \in A$ ).

Giá trị trong mỗi ô là  $q(s, a)$  sẽ được cập nhật liên tục dựa trên trải nghiệm thực tế thông qua công thức:

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_a q(s_{t+1}, a) - q(s_t, a_t)]$$

Qua thời gian, bảng này dần được "lấp đầy", và Agent học được chính sách tối ưu khi lựa chọn tại mỗi trạng thái giá trị  $q$  lớn nhất.

### b. Hạn chế của Tabular Q-learning

Có thể dễ dàng thấy rằng đây là một phương pháp rất hiệu quả trong những môi trường nhỏ, nơi mọi trạng thái và hành động đều có thể được liệt kê rõ ràng. Chỉ cần sau đủ nhiều lần thử thì Q-Table sẽ dần được hoàn thiện và việc còn lại của Agent chỉ cần chọn ra hành động có giá trị lớn nhất. Cách làm này trực quan, dễ hiểu và phù hợp cho các bài toán đơn giản nhưng khi chuyển sang các bài toán thực tế như hệ thống mạng, cách tiếp cận Lookup Table bắt đầu bộc lộ nhiều vấn đề.

#### \* Sự bùng nổ không gian trạng thái:

Vấn đề đầu tiên nằm ở chỗ **Q-learning dạng bảng phụ thuộc hoàn toàn vào khả năng liệt kê trạng thái**. Trong các ví dụ đơn giản như Grid World hay FrozenLake, mỗi trạng thái có thể được đánh số rõ ràng, vì vậy việc tạo bảng là khả thi. Nhưng trong các môi trường thực tế hơn, đặc biệt là trong các hệ thống mạng, một trạng thái có thể bao gồm nhiều thông tin cùng lúc, chẳng hạn như độ trễ, mức sử dụng băng thông, chiều dài hàng đợi, tỷ lệ mất gói, số lượng kết nối đang hoạt động, hay trạng thái của nhiều nút mạng khác nhau. Khi đó, trạng thái thực chất là một vector nhiều chiều, và số tổ hợp có thể xảy ra tăng lên rất nhanh. Nếu vẫn cố biểu diễn bằng bảng, số ô cần lưu sẽ trở nên khổng lồ. Nói cách khác, Q-learning yêu cầu một mức độ "ghi nhớ chi tiết" mà môi trường thực tế không cho phép. Thật sự vậy, nếu

so sánh với nguyên tắc học của con người thì chúng ta cũng **không thể nào ghi nhớ hết mọi thứ** như thế được.

Hệ quả là bộ nhớ để lưu Q-Table trở nên quá lớn, và thời gian cần để Agent đi qua đủ nhiều trạng thái nhằm học được giá trị tin cậy cũng trở nên không thực tế. Trong các môi trường lớn, Agent có thể dành phần lớn thời gian chỉ để "lấp đầy bảng", trong khi nhiều trạng thái quan trọng vẫn chưa từng được thăm dò đủ số lần. Vì vậy, trở ngại không chỉ nằm ở việc lưu trữ, mà còn ở bản chất học tập: tabular methods trở nên quá tốn kém về dữ liệu và thời gian.

**\* Không có khả năng khái quát hóa giữa các trạng thái:**

Trong cách biểu diễn bảng, mỗi trạng thái được xem như một thực thể tách biệt. Nếu Agent từng học tốt ở trạng thái  $s_1$ , điều đó không tự động giúp nó hành động tốt hơn ở trạng thái  $s_2$ , kể cả khi hai trạng thái này rất giống nhau. Đây là khác biệt lớn giữa "ghi nhớ" và "học". Ghi nhớ nghĩa là chỉ biết đúng những gì đã thấy; còn học theo nghĩa mạnh hơn là có thể suy ra hành vi hợp lý ở những tình huống mới nhưng tương tự. Q-learning gần như chỉ làm tốt về đầu tiên. Chẳng hạn, nếu trong một hệ thống mạng, Agent đã học được rằng khi tải mạng ở mức 70% thì nên chọn một hành động điều phối nhất định, thì trực giác cho thấy ở mức 71% hoặc 72% hành vi tốt có lẽ cũng không thay đổi quá nhiều. Tuy nhiên, với Q-Table, các trạng thái này vẫn là những ô hoàn toàn khác nhau. Agent không nhìn thấy sự tương đồng ấy, nên phải học lại gần như từ đầu cho từng trường hợp. Chính điều này làm cho việc học trở nên chậm, kém linh hoạt và khó mở rộng.

Từ đây xuất hiện một vấn đề rất thực tế: **Q-learning xử lý trạng thái liên tục rất kém**. Trong nhiều bài toán, trạng thái không nằm trên một tập hữu hạn mà thay đổi liên tục. Ví dụ, độ trễ có thể là 10.1 ms, 10.2 ms, 10.3 ms; thông lượng có thể dao động liên tục; mức chiếm dụng hàng đợi cũng vậy. Q-learning dạng bảng không thể làm việc trực tiếp với các giá trị liên tục

này, nên người thiết kế thường phải rời rạc hóa chúng thành các khoảng. Tuy nhiên, rời rạc hóa luôn kéo theo đánh đổi. Nếu chia quá thô, ta làm mất nhiều thông tin quan trọng; nếu chia quá mịn, số lượng trạng thái lại phình to, quay trở về bài toán bùng nổ kích thước. Nói cách khác, tabular methods buộc người thiết kế phải lựa chọn giữa **mất chi tiết** và **khó học**, và không có lựa chọn nào thực sự tối ưu cho các môi trường phức tạp.

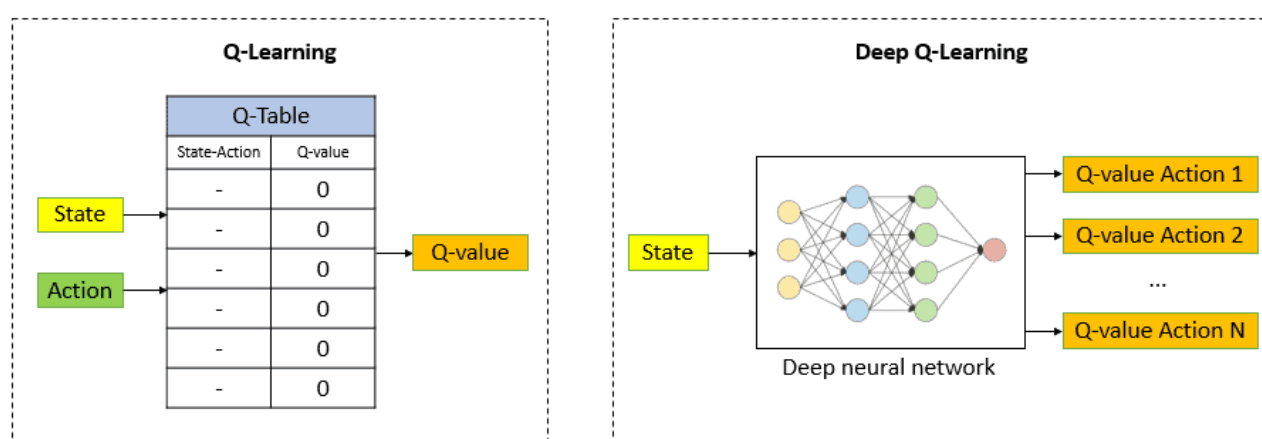
Một khó khăn khác là **hiệu quả sử dụng dữ liệu thấp**. Vì mỗi ô trong bảng được học gần như độc lập, thông tin thu được từ một trải nghiệm thường chỉ giúp cải thiện một phần rất nhỏ của toàn bộ không gian trạng thái-hành động. Trong môi trường nhỏ, điều này không phải vấn đề lớn; nhưng trong môi trường lớn, nó khiến việc học trở nên chậm chạp. Agent cần rất nhiều mẫu để cập nhật đủ nhiều ô khác nhau trong bảng. Điều này đặc biệt bất lợi trong những tình huống mà việc thu thập dữ liệu tốn kém, hoặc khi môi trường thay đổi theo thời gian. Trong một hệ thống mạng thực tế, điều kiện lưu lượng có thể biến động liên tục, nên nếu mô hình học quá chậm, kiến thức vừa học xong có thể đã không còn phù hợp.

## 2. Giới thiệu về Deep Q-Network

### a. Ý tưởng hình thành

Mặc dù bộc lộ nhiều điểm yếu, nhưng vẫn phải công nhận rằng, ý tưởng cập nhật giá trị của Q-learning vẫn rất mạnh mẽ và hoàn toàn không có vấn đề. Điểm cần phải cải tiến duy nhất là cách biểu diễn các  $q(s, a)$  để phù hợp với các bài toán lớn hơn. Từ nhu cầu đó, một ý tưởng mới xuất hiện: thay vì lưu từng giá trị  $q(s, a)$  trong bảng, ta có thể dùng một **hàm xấp xỉ** để dự đoán giá trị này. Nếu hàm xấp xỉ được học tốt, thì hai trạng thái tương tự nhau có thể cho ra các giá trị  $q$  tương tự nhau, dù Agent chưa từng gặp chính xác trạng thái đó trước đây. Nói cách khác, mô hình bắt đầu có **khả năng generalisation**. Đây chính là điều mà tabular methods còn thiếu.

Ban đầu, người ta từng thử các hàm xấp xỉ đơn giản như **mô hình tuyến tính**. Tuy nhiên, trong các môi trường có cấu trúc phức tạp, quan hệ giữa trạng thái và giá trị hành động thường là phi tuyến mạnh, nên những mô hình đơn giản này khó biểu diễn đầy đủ. Sự phát triển của Deep Learning đã tạo ra một bước ngoặt quan trọng: mạng nơ-ron sâu có khả năng học các ánh xạ phi tuyến phức tạp, trích xuất đặc trưng từ dữ liệu đầu vào và khái quát tốt hơn trên không gian trạng thái lớn. Khi kết hợp tư tưởng cập nhật của Q-learning với sức mạnh biểu diễn của mạng nơ-ron sâu, ta đi đến ý tưởng cốt lõi của **Deep Q-Network (DQN)**.



Hình 1. Từ Q-learning đến DQN<sup>1</sup>

DQN được đề xuất bởi [DeepMind](https://www.baeldung.com/cs/q-learning-vs-deep-q-learning-vs-deep-q-network) vào năm 2013 với dự định ban đầu để Agent có thể học trực tiếp từ dữ liệu đầu vào có độ phức tạp cao như các raw pixels trong các trò chơi Atari. Đây là sự thành công góp phần tạo nền tảng lý thuyết vững chắc và có khả năng ứng dụng RL vào trong các bài toán thực tế.

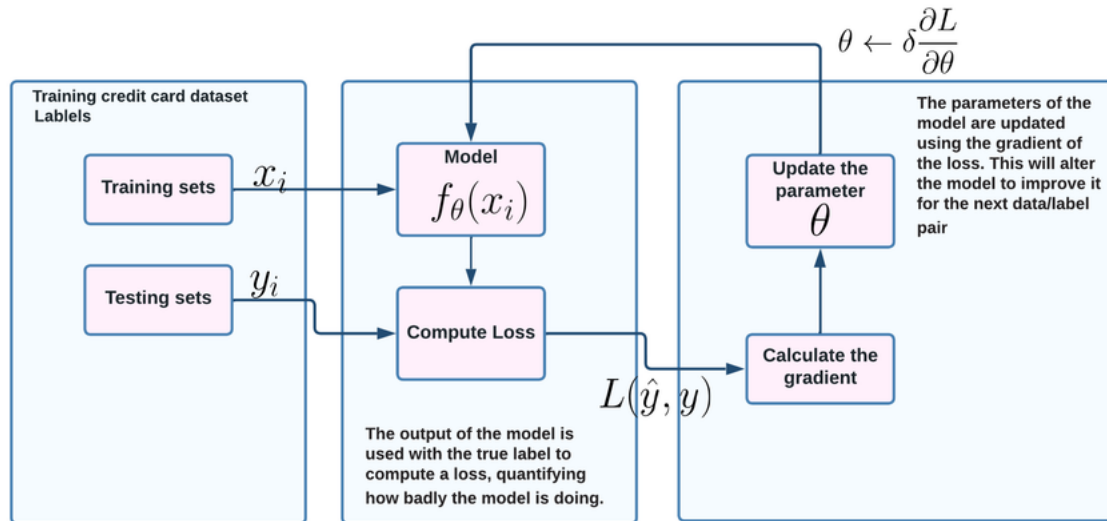
### b. Cách thức hoạt động

Về cơ bản, DQN là một phương pháp kết hợp giữa Q-learning và Deep Learning, trong đó mạng nơ-ron được sử dụng để **xấp xỉ hàm giá trị hành động**. Khi ấy, mục tiêu không còn là ghi nhớ một ô cho mỗi cặp trạng thái-hành động, mà là học một ánh xạ tổng quát:

$$Q(s, a) \approx f_{\theta}(s, a)$$

<sup>1</sup> <https://www.baeldung.com/cs/q-learning-vs-deep-q-learning-vs-deep-q-network>

trong đó  $f_\theta$  là một hàm có tham số. Cách tiếp cận này mang lại một thay đổi rất quan trọng về bản chất. Nhờ khả năng dự đoán của mạng nơ-ron, không chỉ giúp giảm phụ thuộc vào việc liệt kê trạng thái, mà còn cho phép mô hình học được mối quan hệ giữa các trạng thái, từ đó đưa ra dự đoán hợp lý cho những tình huống chưa từng gặp.



Hình 2. Workflow tổng quan của kiến trúc Deep Learning<sup>2</sup>

Để hiểu rõ hơn cách DQN vận hành, trước hết cần liên hệ với cách mà một mô hình nơ-ron hoạt động trong DL thông thường. Trong supervised learning, một mạng nơ-ron học một ánh xạ từ đầu vào  $x$  đến đầu ra  $y$ , thông qua việc so sánh giữa giá trị dự đoán và nhãn thật. Quá trình học xoay quanh việc giảm sai số giữa *prediction* và *target* từ prediction network và target network. DQN kế thừa chính cơ chế này, nhưng thay vì có sẵn nhãn đúng, nó tự xây dựng **target** từ dữ liệu tương tác với môi trường. Điều này khiến DQN trở thành một dạng học lai, vừa mang đặc trưng của supervised learning, vừa mang bản chất của reinforcement learning.

<sup>2</sup> <https://www.researchgate.net/publication/379945143/figure/fig2/AS:11431281431976794@1746829931451/Workflow-of-deep-learning-architectures.tif>



Trong DQN, mạng nơ-ron nhận trạng thái  $s$  làm đầu vào và xuất ra một vector gồm các giá trị  $q$  cho tất cả các hành động khả dĩ. Ví dụ, nếu Agent có hai hành động “trái” và “phải”, đầu ra của mạng có thể là một vector như  $[1.5, 2.7]$ . Điều này có nghĩa là, tại trạng thái hiện tại, nếu chọn hành động “trái” thì giá trị kỳ vọng dài hạn là 1.5, còn nếu chọn “phải” thì là 2.7. Quá trình học của DQN sẽ xoay quanh hai khái niệm quan trọng: prediction và target. Prediction là giá trị mà mạng hiện tại dự đoán cho hành động đã thực hiện. Chẳng hạn, nếu Agent chọn hành động thứ hai, thì prediction chính là 2.7. Đây là “niềm tin hiện tại” của mô hình về mức độ tốt của hành động đó. Trong khi đó, target là giá trị mà prediction nên tiến tới, được xây dựng từ phần thưởng thực tế và trạng thái tiếp theo. Nếu Agent nhận được phần thưởng  $r$  và chuyển sang trạng thái  $s'$ , ta có thể ước lượng target bằng công thức:

$$y = r + \gamma \underbrace{\max_{a'}}_{a'} Q(s', a')$$

Ý nghĩa của biểu thức này là: giá trị của hành động hiện tại không chỉ phụ thuộc vào phần thưởng ngay lập tức, mà còn phụ thuộc vào khả năng đạt được phần thưởng trong tương lai từ trạng thái tiếp theo. Và cuối cùng, để có thể điều chỉnh sao cho prediction dần tiến gần đến target thì phải thông qua 2 phần: ước lượng sự sai khác giữa prediction và target ; và điều chỉnh để loss giảm theo thời gian. Đầu tiên, để ước lượng sự sai khác giữa 2 phần này sẽ phải thông qua hàm mất mát (*loss function*), thường là **Mean Squared Error (MSE)**. Thứ hai, để điều chỉnh các trọng số sao cho loss giảm xuống, điều này được thực hiện thông qua cơ chế lan truyền ngược (**backpropagation**) kết hợp với **gradient descent**.

Tóm lại, DQN hoạt động hiệu quả nhờ là việc sử dụng hai mạng nơ-ron: main network và target network. Main network được cập nhật liên tục và dùng để đưa ra prediction. Target network là một bản sao của main network, nhưng được cập nhật chậm hơn và dùng để tính target. Việc tách hai mạng này giúp tránh tình trạng target thay đổi quá nhanh trong quá trình học, từ đó làm cho việc tối ưu trở nên ổn định hơn. Có thể hình dung main network là



“người học”, còn target network là “đáp án tham chiếu” được giữ cố định trong một khoảng thời gian ngắn.

### c. Hiện thực thuật toán

Nếu trong Q-learning, quá trình huấn luyện diễn ra khá trực tiếp - mỗi bước tương tác với môi trường sẽ dẫn đến một lần cập nhật ngay giá trị  $q(s, a)$ , thì trong DQN, toàn bộ workflow đã được thay đổi đáng kể để phù hợp với việc sử dụng mạng nơ-ron. Những thay đổi này không làm thay đổi nguyên lý Bellman, nhưng làm thay đổi cách dữ liệu được sử dụng và cách cập nhật được thực hiện.

---

#### Algorithm 1 Deep Q-Network (DQN)

---

```

1: Initialise replay buffer  $\mathcal{D}$ ; main network parameters  $\theta$ ; target network parameters  $\theta^- \leftarrow \theta$ 
2: for each episode do
3:   Observe initial state  $s$ 
4:   for each step do
5:     Select action  $a$  using  $\epsilon$ -greedy from  $\hat{q}(s, a, \theta)$ 
6:     Execute  $a$ , observe  $r, s'$ , and terminal flag
7:     Store transition  $(s, a, r, s')$  in  $\mathcal{D}$ 
8:     Sample mini-batch  $(s_j, a_j, r_j, s'_j)$  from  $\mathcal{D}$ 
9:     Compute targets:  $y_j = \begin{cases} r_j, & \text{if } s'_j \text{ is terminal} \\ r_j + \gamma \max_{a'} \hat{q}(s'_j, a', \theta^-), & \text{otherwise} \end{cases}$ 
10:    Update  $\theta$  by minimising  $\frac{1}{B} \sum_{j=1}^B (y_j - \hat{q}(s_j, a_j, \theta))^2$ 
11:     $s \leftarrow s'$ 
12:    if target update step then
13:       $\theta^- \leftarrow \theta$ 
14:    end if
15:    if terminal then
16:      break
17:    end if
18:  end for
19: end for

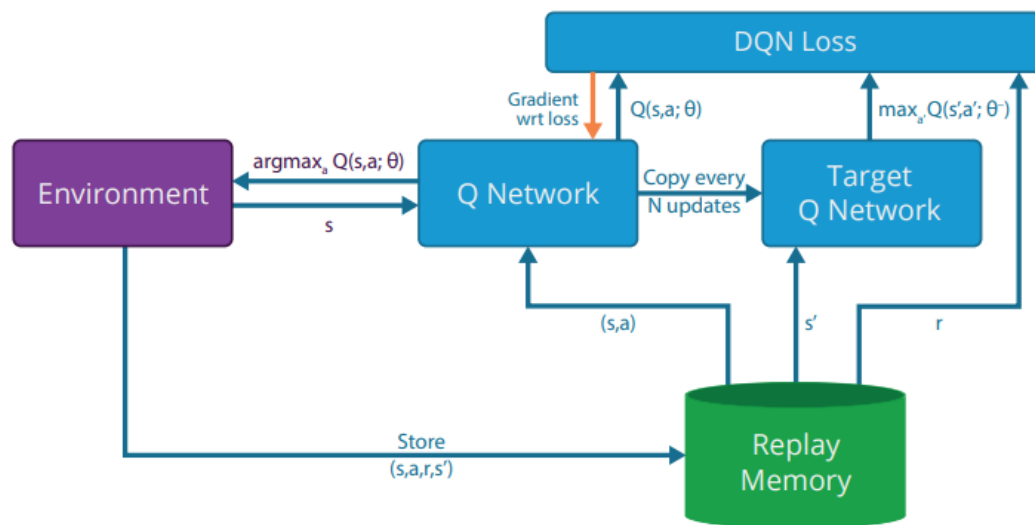
```

---

Hình 3. DQN pseudocode

Điểm khác biệt đầu tiên và quan trọng nhất là DQN không còn cập nhật ngay sau mỗi bước tương tác. Trong DQN, sau khi tương tác thì trải nghiệm  $(s, a, r, s')$  không được dùng ngay mà được lưu vào một bộ nhớ gọi là **experience replay buffer**. Điều này tạo ra một sự thay đổi lớn: quá trình học được tách khỏi dòng thời gian của môi trường. Vai trò của experience replay xuất phát từ một vấn đề cốt lõi mà Q-learning dạng bảng không gặp phải:

tính phụ thuộc của dữ liệu khi dùng **neural network**. Trong Q-learning, mỗi cập nhật chỉ ảnh hưởng đến một ô trong bảng, nên việc dữ liệu có tương quan không gây vấn đề nghiêm trọng. Ngược lại, trong DQN, một lần cập nhật trọng số có thể ảnh hưởng đến nhiều trạng thái khác nhau. Nếu dữ liệu đầu vào là các trạng thái liên tiếp (có tính tương quan cao), mạng rất dễ học lệch hoặc dao động. Experience replay giải quyết điều này bằng cách lưu lại nhiều trải nghiệm và cho phép lấy mẫu ngẫu nhiên, giúp dữ liệu huấn luyện trở nên đa dạng và gần với giả định độc lập. Ngoài ra, nó còn cho phép **tái sử dụng dữ liệu nhiều lần**, thay vì chỉ dùng một lần như trong Q-learning.



Hình 4. Kiến trúc DQN<sup>3</sup>

Một thay đổi quan trọng thứ hai là cách cập nhật không còn dựa trên một mẫu duy nhất, mà dựa trên **mini-batch**. Trong Q-learning, mỗi lần cập nhật chỉ sử dụng một cặp  $(s, a)$ . Trong DQN, ta lấy một tập nhỏ các trải nghiệm từ replay buffer và tính toán loss trên toàn bộ tập này. Điều này giúp việc cập nhật trở nên ổn định hơn, vì sai số không còn phụ thuộc vào một mẫu đơn lẻ mà được trung bình hóa. Đồng thời, mini-batch updates cũng phù hợp hơn với cách huấn luyện mạng nơ-ron, đặc biệt khi sử dụng GPU.

<sup>3</sup> <https://kaustabpal.github.io/notes/rl/>

Và điểm đáng chú ý cuối cùng, target network không được cập nhật ngay lập tức, mà chỉ được đồng bộ với main network sau một số bước nhất định. Nhờ đó, giá trị target được giữ tương đối ổn định trong một khoảng thời gian, giúp quá trình tối ưu dễ hội tụ hơn.

### 3. Giới thiệu về PyTorch

[PyTorch](#) là một thư viện học sâu mã nguồn mở do Facebook AI Research (FAIR) phát triển và được công bố vào năm 2016. Thư viện này được xây dựng nhằm thay thế và cải tiến từ Torch (viết bằng Lua), chuyển sang nền tảng Python để phù hợp hơn với cộng đồng nghiên cứu và phát triển trí tuệ nhân tạo. Nhờ cú pháp rõ ràng, gần gũi với lập trình Python thông thường, PyTorch nhanh chóng trở thành một trong những công cụ được sử dụng rộng rãi trong các nghiên cứu về học máy, đặc biệt là học tăng cường.

Một trong những đặc điểm quan trọng của PyTorch là cơ chế tính toán đồ thị động (dynamic computation graph). Khác với một số framework trước đây yêu cầu xây dựng toàn bộ đồ thị tính toán trước khi chạy (static graph), PyTorch cho phép định nghĩa và thay đổi cấu trúc mô hình ngay trong quá trình thực thi. Điều này đặc biệt hữu ích trong các bài toán học tăng cường, nơi mà môi trường và dữ liệu có thể thay đổi liên tục theo từng bước tương tác.

Bên cạnh PyTorch, một số công cụ tương đương có thể kể đến như **TensorFlow (Google)**, **Keras** (nay tích hợp trong TensorFlow), hay **JAX**. TensorFlow thường được sử dụng trong các hệ thống sản xuất lớn nhờ hệ sinh thái mạnh mẽ, trong khi Keras nổi bật với sự đơn giản khi xây dựng mô hình ở mức cao. JAX lại tập trung vào tối ưu hóa hiệu năng và tính toán số học. Tuy nhiên, PyTorch thường được ưu tiên trong nghiên cứu và giảng dạy vì tính linh hoạt, dễ hiểu và dễ triển khai các ý tưởng mới.

Để hỗ trợ việc xây dựng các mô hình học sâu, PyTorch cung cấp một số thành phần quan trọng:

- **Tensor**: là cấu trúc dữ liệu cơ bản, tương tự như mảng nhiều chiều. Tensor được sử dụng để biểu diễn dữ liệu đầu vào, trạng thái môi trường hoặc tham số của mô hình. Một ưu điểm lớn là tensor có thể dễ dàng chuyển đổi giữa CPU và GPU để tăng tốc tính toán.
- **Neural Network (torch.nn)**: cung cấp các công cụ để xây dựng mạng nơ-ron như các lớp tuyến tính, hàm kích hoạt và cấu trúc mô hình.
- **Autograd**: là cơ chế tự động tính toán gradient, giúp cập nhật trọng số của mạng một cách tự động trong quá trình huấn luyện. Người học không cần phải tự triển khai các phép tính đạo hàm phức tạp.
- **Optimizer**: hỗ trợ các thuật toán tối ưu như SGD hay Adam để điều chỉnh tham số của mô hình dựa trên gradient. Điều này giúp quá trình học diễn ra ổn định và hiệu quả hơn.
- **Hỗ trợ GPU**: PyTorch cho phép tận dụng sức mạnh của GPU để tăng tốc quá trình huấn luyện, đặc biệt quan trọng khi làm việc với mạng nơ-ron có nhiều tham số hoặc dữ liệu lớn.

#### 4. Giới thiệu về Stable-Baselines3

[Stable-Baselines3 \(SB3\)](#) là một thư viện mã nguồn mở dành cho học tăng cường, được phát triển dựa trên nền tảng của dự án Stable-Baselines trước đó. Phiên bản thứ ba này được xây dựng lại hoàn toàn bằng PyTorch (thay vì TensorFlow như các phiên bản cũ), với mục tiêu cung cấp một bộ công cụ đơn giản, nhất quán và dễ sử dụng để triển khai các thuật toán học tăng cường hiện đại. SB3 được cộng đồng nghiên cứu và thực hành đánh giá cao nhờ tính ổn định và khả năng tái sử dụng trong nhiều bài toán khác nhau.

Về khả năng, Stable-Baselines3 cung cấp sẵn nhiều thuật toán học tăng cường phổ biến như DQN, PPO, A2C, SAC, TD3,... với các cài đặt đã được kiểm chứng. Người dùng có thể

nhanh chóng huấn luyện một Agent mà không cần phải xây dựng lại toàn bộ thuật toán từ đầu. Ngoài ra, thư viện này tích hợp chặt chẽ với **Gymnasium**, cho phép dễ dàng kết nối với các môi trường mô phỏng tiêu chuẩn. SB3 cũng hỗ trợ các chức năng quan trọng như lưu/khôi phục mô hình, đánh giá hiệu năng, logging kết quả và tùy chỉnh chính sách (policy) hoặc kiến trúc mạng.

Về điểm mạnh, Stable-Baselines3 nổi bật ở tính dễ sử dụng, tính ổn định và khả năng tái lập kết quả (reproducibility). Các thuật toán trong SB3 được triển khai theo các tiêu chuẩn tốt nhất (best practices), giúp đảm bảo kết quả huấn luyện nhất quán hơn so với việc tự cài đặt từ đầu. Một điểm đáng chú ý là Stable-Baselines3 được thiết kế theo hướng chuẩn hóa quy trình huấn luyện, giúp giảm thiểu các lỗi phổ biến khi triển khai học tăng cường.

Bên cạnh Stable-Baselines3, một số công cụ tương đương có thể kể đến như **RLlib** (thuộc Ray, mạnh về huấn luyện phân tán), **CleanRL** (tập trung vào các cài đặt đơn giản, dễ đọc), **Tianshou** (linh hoạt và hiệu năng cao), hay **Acme** (do DeepMind phát triển, thiên về nghiên cứu). Tuy nhiên, SB3 thường được lựa chọn trong giảng dạy và thực hành cơ bản nhờ giao diện thân thiện, tài liệu rõ ràng và khả năng tích hợp tốt với các môi trường chuẩn.

## C. THỰC HÀNH

### 1. Thực hiện tạo ra một mạng nơ-ron đơn giản bằng PyTorch

**Mô tả:** Sinh viên tìm hiểu về các khái niệm của PyTorch và tiến hành hiện thực 1 mạng nơ-ron SimpleNetwork.

- **Bước 1:** Cài đặt thư viện liên quan: `pip install -r requirements.txt`
- **Bước 2:** Mở file **Lab3.1-st.ipynb**, thực hiện chạy *Part 1* đã được cung cấp sẵn.

Tạo 2 tensor 1 chiều cho phép chế độ tính gradient để phục vụ cho việc tính toán Automatic Differentiation. Các tensor này có giá trị lần lượt là [1.0, 2.0, 3.0] và [4.0, 5.0, 6.0]

```
# Create tensors with gradient tracking to enable automatic differentiation
# values: [1.0, 2.0, 3.0] and [4.0, 5.0, 6.0]
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = torch.tensor([4.0, 5.0, 6.0], requires_grad=True)
```

Tính toán tự động gradient với Autograd hàm:  $y = x^2$

```
...
# Forward pass
y = x ** 2 # y = x^2
z = y.sum() # z = sum(y)
...
# Backward pass (compute gradients)
z.backward() # Computes dz/dx
print(f"\nGradients (dz/dx): {x.grad}")
print(f"Expected: [4.0, 6.0] (since d/dx[x^2] = 2x)")
```

Định nghĩa một mạng nơ-ron đơn giản: có 2 lớp, ở giữa là hàm kích hoạt ReLU. Tổng quát

kiến trúc: Input (4) -> Linear -> Hidden (64) -> ReLU -> Linear -> Output (2)

```
def __init__(self, input_size=4, hidden_size=64, output_size=2):
    ...
    super(SimpleNet, self).__init__()
    self.fc1 = nn.Linear(input_size, hidden_size) # Input to hidden
    self.relu = nn.ReLU() # Activation
    self.fc2 = nn.Linear(hidden_size, output_size) # Hidden to output

def forward(self, x):
    ...
    x = self.fc1(x) # Linear transformation
    x = self.relu(x) # Apply ReLU activation
    x = self.fc2(x) # Output layer
    return x
```

Thực hiện vòng lặp huấn luyện cơ bản theo quy trình: dự đoán → tính loss → lan truyền ngược → cập nhật trọng số.

```
# Create network and optimizer
net = SimpleNet(input_size=4, hidden_size=64, output_size=2)
optimizer = optim.Adam(net.parameters(), lr=0.01) # Adam optimizer
loss_fn = nn.MSELoss() # Mean Squared Error loss
...
for epoch in range(5):
    # 1. Forward pass: compute predictions
```

```

predictions = net(X_train)

# 2. Compute loss
loss = loss_fn(predictions, y_train)

# 3. Backward pass: compute gradients
optimizer.zero_grad() # Reset gradients from previous iteration
loss.backward()        # Backpropagation: compute all gradients

# 4. Update weights
optimizer.step()        # Gradient descent step

```

**?** Gradient của một hàm số là gì? Chúng có ý nghĩa gì đối với việc cập nhật trọng số sau mỗi lần học tập trong mạng nơ-ron?

## 2. Hiện thực DQN trên môi trường có sẵn

**Mô tả:** Sinh viên hiện thực các thành phần của DQN, đặc biệt là sử dụng PyTorch để hiện thực phần lõi mạng nơ-ron của DQN đối với môi trường FrozenLake-v1 có sẵn trên Gymnasium.

- **Bước 1:** Tiếp tục với file **Lab3.1-st.ipynb**, thực hiện chạy *Part 2* đã được cung cấp sẵn.
- **Bước 2:** Tạo bộ nhớ trải nghiệm:

Lưu trữ các trải nghiệm (*state, action, reward, next\_state, done*) để Agent có thể học lại từ quá khứ, giúp quá trình học ổn định hơn và tránh bị lệ thuộc vào dữ liệu liên tiếp.

```

class ReplayBuffer:
    ...
    def push(self, state, action, reward, next_state, done):
        self.buffer.append((state, action, reward, next_state, done))
    def sample(self, batch_size):
        ...
        # Sample a random batch of transitions

```

- **Bước 3:** Tạo cấu trúc mạng nơ-ron DQN. Kiến trúc của mạng tương tự với mạng nơ-ron ở trên.

**?** Xem lại lại về state của môi trường FrozenLake-v1, hãy phân tích về cấu trúc input và output của mạng nơ-ron DQN cần phải thiết kế như thế nào là hợp lý?



```
class DQNNetwork(nn.Module):
    def __init__(self, state_size=16, action_size=4):
        ...
        self.fc1 = nn.Linear(state_size, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, action_size)
    def forward(self, state):
        ...
```

- **Bước 4:** Tạo DQN Agent điều phối toàn bộ quá trình như: chọn hành động, lưu trữ trải nghiệm, huấn luyện mạng và điều chỉnh tỷ lệ khám phá.

```
class DQNAgent:
    ...
    def select_action(self, state, training=True):
        # Select action using epsilon-greedy policy
    def store_transition(self, state, action, reward, next_state, done):
        # Store transition in replay buffer
    def train_step(self, batch_size=32):
        # Update Q-network with one mini-batch from replay buffer
    def decay_epsilon(self):
        # Decay exploration rate at the end of each episode
```

- **Bước 5:** Thực hiện chạy huấn luyện và tiến hành đánh giá sau 500 episodes.

**?** Bạn có nhận xét gì về kết quả đạt được? Với kết quả đó, theo bạn thì những yếu tố nào có thể ảnh hưởng đến kết quả thu được?

## D. YÊU CẦU & NỘI BÀI

### 1. Yêu cầu

1. *Đối với FrozenLake Environment*, hoàn thành các block code định nghĩa tại file **Lab3.1-st.ipynb**, **Phần 2**. Giải thích ngắn về cơ chế dùng để cập nhật tham số: gradient  $\rightarrow$  loss  $\rightarrow$  backpropagation  $\rightarrow$  optimizer, tương ứng trong các đoạn mã. Thực hiện vẽ biểu đồ loss theo epoch và nhận xét tốc độ hội tụ và dao động theo từng trường hợp:

- Optimizer SGD (lr=0.01)
- Learning rate: 1e-2, 1e-3, 5e-4

**Lưu ý:** Chạy lại với nhiều seed khác nhau và tổng hợp kết quả ( $\geq 5$ ).

2. Đối với *VacuumCleaner Environment*, dựa trên kiến trúc và quy trình tích hợp DQN Agent vào môi trường ở nội dung trước, thực hiện phân tích thiết kế state hiện tại để có thể tích hợp DQN vào tại file **Lab3.2-st.ipynb, Phần 1**. Sau khi ứng dụng kiến trúc và các tham số tương ứng ở nội dung trước vào VacuumCleaner Environment, hãy phân tích kết quả so với chính sách dựa trên Q-learning đã được hiện thực tại Lab02 về: reward, tỷ lệ hoàn thành dọn dẹp, số bước robot đã di chuyển trên cùng 1 số lượng episode huấn luyện.

3. Hãy thử cải tiến môi trường và các yếu tố của DQN Agent để mạng lại kết quả tốt hơn. Một số nội dung có thể cân nhắc:

- Thiết kế lại reward: phạt va tường / lấp ô cũ mạnh hơn; bonus khi hoàn tất toàn bộ
- Cải tiến cấu trúc Q-network: hidden size: 64 / 128 / 256 → Tăng layer
- Thay đổi các bộ trọng số: learning rate, batch\_size 32 / 64, epsilon\_decay 0.99 / 0.995 / 0.998

Hãy báo cáo lại toàn bộ quá trình cải tiến của nhóm để đạt được **success rate > 80%**.

4. Đối với *LoadBalancing Environment*, thực hiện phân tích thiết kế state hiện tại để có thể tích hợp DQN vào tại file **Lab3.2-st.ipynb, Phần 2**. Hiện thực chính sách **Round Robin** để thực hiện quá trình đánh giá.

5. Sau khi huấn luyện với DQN, thực hiện chạy môi trường với chính sách dựa trên DQN và Round Robin trong 100 episode. Viết hàm lưu trữ các metrics vào file *.csv*, các metrics cần lưu bao gồm: tổng thời gian chạy, tổng tác vụ đã tạo ra, số tác vụ được tiếp nhận để xử lý, số lượng tác vụ bị bỏ qua, số lượng tác vụ đã hoàn thành, phần thưởng, tổng queue time trung bình, độ lệch tải giữa các servers, tần suất chọn từng server. Vẽ bảng tổng hợp so sánh và phân tích giữa 2 chính sách.

6. Thiết kế lại reward để cân bằng các yếu tố: queue time, độ lệch tải và tần suất chọn servers. Thực hiện huấn luyện lại và tiến hành đánh giá sự tác động của reward mới với kết quả của chính sách mới dựa trên DQN và chính sách cũ.

7. Thực hiện điều chỉnh *VacuumCleaner Environment* hiện tại để có khả năng tích hợp DQN được cung cấp bởi Stable-Baselines3: scale reward, state,... Thực hiện huấn luyện và tìm ra bộ trọng số để đạt được **success rate > 70%**.

## 2. Yêu cầu nộp bài

- Sinh viên tìm hiểu và thực hành theo hướng dẫn. Thực hiện **theo nhóm**.
- Sinh viên báo cáo kết quả thực hiện và nộp bài bằng file. Trong đó:
  - Trình bày chi tiết quá trình thực hành và trả lời các câu hỏi nếu có (kèm theo các ảnh chụp màn hình tương ứng).
  - Giải thích các kết quả đạt được.
  - Tải mẫu báo cáo thực hành và trình bày theo mẫu được cung cấp.

Nén tất cả các file và đặt tên file theo định dạng theo mẫu:

**NhomY-LabX\_MSSV1\_MSSV2**

Ví dụ: Nhom1-Lab03\_29520001\_29520002\_29520003

- Nộp báo cáo trên theo thời gian đã thống nhất tại website môn học.
- Các bài nộp không tuân theo yêu cầu sẽ **KHÔNG** được chấm điểm.

**HẾT**

Chúc các bạn hoàn thành tốt bài thực hành!